

Contents

技术报告：手语桥（**SignBridge**）端侧手语识别终端

1、信息表	1
2、摘要	1
3、正文	2
3.1 绪论	2
3.2 系统方案设计	2
3.3 核心算法与技术原理	3
3.4 系统实现	4
3.5 系统测试与结果分析	7
3.6 AI-Native 开发说明	7
3.7 总结与展望	8

技术报告：手语桥（**SignBridge**）端侧手语识别终端

1、信息表

项目	内容
作品名称	手语桥（ SignBridge ）端侧手语识别终端
队伍名称	zuoyeyushufengzhou
团队分工	周宇翔（全栈开发）
选题方向	新硬件平台适配 + AI 硬件产品创新

2、摘要

手语桥是运行在 ESP32-P4-Function-EV-Board 上的离线手语双向翻译终端，基于 openvela（Apache NuttX）系统。针对听障群体日常沟通场景，通过 MIPI-CSI 摄像头采集手部画面，在本地完成 21 关键点提取与时序分类，识别结果以文字显示于 7 寸触摸屏并同步播放语音；反向通路为麦克风采集、语音活动检测与手语动画回放。全部推理与媒体处理在端侧完成，无云端依赖。

主要成果：

- 完成 ESP32-P4-Function-EV-Board 在 openvela 上的首次板级移植（新硬件适配赛道）：芯片架构、espressif HAL（80+ 外设）、三块板级 目录、35 个外设验证配置
- 自研 MIPI-CSI 驱动：DW-GDMA 写入 PSRAM 双缓冲，帧回调交付上层；MIPI-DSI 显示链路（EK79007 1024×600）与 GT911 触摸驱动增强（I2C 重试、总线恢复、事件统计）
- 完成 ESP32-P4 的 CMake 构建支持（多链接脚本、esp-hal 补丁、mkimage 出 bin），./build.sh --cmake 全链路产出 nuttx.bin
- 应用双线交付：signbridge 推理应用（本仓库，含 TFLM 推理链路、状态机、LVGL 界面、语音播报、VAD、离线纠错）与 lvgldemo UI/摄像头预览演示（open-vela/nuttx-apps PR #123）
- 当前状态：编译、静态检查、引脚核对全部通过；整机链路（烧录后 UI/语音/推理联调）基本完成，待烧录复验

3、正文

3.1 绪论

1. 项目背景与问题定义 应用场景：听障群体的日常沟通场景，包括家庭交流、公共场所服务、教育场景等。

行业痛点：

- 现有手语翻译方案多依赖云端，存在网络延迟和隐私泄露风险
- 专业手语翻译人员稀缺，成本高昂
- 现有手语识别设备体积大、功耗高，不便于携带

用户痛点：

- 听障人士与健听人士沟通困难，依赖手语翻译员或文字交流
- 现有手语识别 APP 需要持续联网，离线场景无法使用
- 识别延迟影响沟通流畅度

2. 技术难点

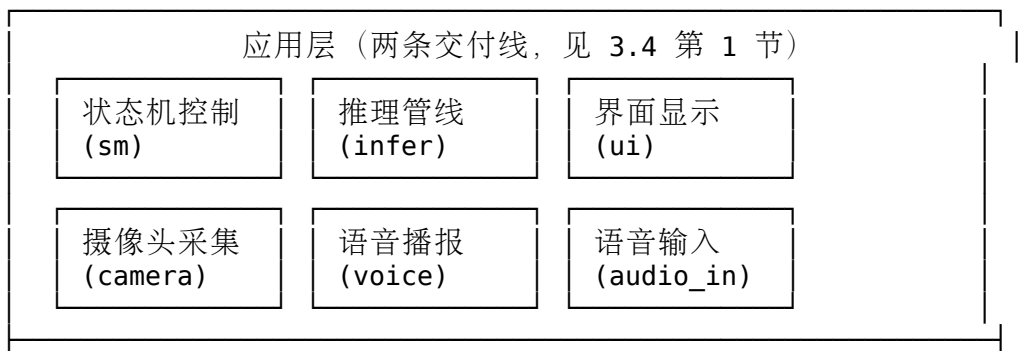
- 端侧算力限制：ESP32-P4 双核 RISC-V 400MHz，需在有限算力下 实现实时推理
- 多设备协同：摄像头、麦克风、显示屏、扬声器需高效协同工作
- 低功耗设计：便携设备需长时间续航，需精细的功耗管理
- 驱动适配：ESP32-P4-Function-EV-Board 在 openvela 上无现成 支持，需从零移植（含构建系统）

3. 创新点

- 技术层面：
 - ESP32-P4 首次在 openvela 上完成板级移植并打通 CMake 构建链路
 - 端侧推理全链路设计：摄像头帧采集 → 手部关键点检测 → 时序分类 → 文字输出
 - 双向翻译：手语 → 文字 + 语音 / 语音 → 手语动画
- 场景层面：
 - 纯离线运行，无网络依赖，保护用户隐私
 - 便携式设计，适合日常携带使用
- 交互层面：
 - 7 寸触摸屏实时显示识别结果和摄像头预览
 - 语音播报提供即时反馈
 - 手语动画实现双向沟通

3.2 系统方案设计

1. 系统总体架构



中间件层
LVGL · TFLite Micro · nxplayer · ROMFS
驱动层
EK79007(MIPI DSI) · GT911(touch) · ES8311(audio) · SC2336(camera) · I2S/I2C/FB/input
板级支持
esp32p4-function-ev-board · openvela/NuttX

数据流:

1. 摄像头采集: SC2336 → MIPI CSI → DW-GDMA → PSRAM 双缓冲 → 帧回调
2. 推理处理: 帧 → TFLite Micro(hand landmark) → 21×3 关键点 → 32 帧滑动窗口 → INT8 MLP 分类 → 类别 + 置信度
3. 结果输出: 分类结果 → LVGL 显示 + nxplayer 语音播报
4. 语音输入: 麦克风 → I2S → VAD → 唤醒兜底

2. 方案论证与选型 开发板选型理由:

- ESP32-P4: 双核 RISC-V 400MHz, 32MB PSRAM, 满足端侧推理算力需求
- 集成 MIPI DSI/CSI 接口, 支持 7 寸高清显示屏和摄像头
- WiFi6/BLE5 支持, 预留云端扩展能力
- 功耗控制能力适合便携设备

端/云职责划分:

- 端侧: 摄像头采集、推理识别、界面显示、语音播报、语音输入
- 云端 (预留): 语义纠错与补全、模型更新
- 降级策略: 断网时完全离线运行, 云端纠错功能降级为本地置信度过滤

与备选方案对比:

方案	优点	缺点	选择理由
ESP32-P4 + openvela	算力充足、外设丰富、开源生态	需要板级移植	首次适配, 展示移植能力
树莓派 + Linux	生态成熟、开发便捷	功耗高、体积大、成本高	不适合便携场景
手机 APP + 云端	开发简单、算力强	依赖网络、隐私风险	不满足离线需求

3. 关键模块设计 通信模块:

- I2C0 单总线挂三设备 (触摸/音频/摄像头), 400kHz, NuttX 总线锁
- MIPI DSI 2 lane 显示接口
- MIPI CSI 2 lane 摄像头接口 (自研驱动, 见 3.4)
- I2S0 音频接口 (麦克风输入 + 扬声器输出)

交互模块:

- 7 寸触摸屏: 1024×600 分辨率, GT911 轮询模式
- LVGL 界面框架: 横屏布局, 左侧摄像头预览, 右侧识别结果
- 语音交互: 麦克风采集 + 语音活动检测 + 语音播报

3.3 核心算法与技术原理

1. AI 算法实现 端侧模型:

- 手部关键点检测: MediaPipe Hand Landmark (TFLite 量化模型)
 - 输入: 256×256 RGB 图像
 - 输出: 21×3 关键点坐标 (x, y, z)
 - 模型大小: 约 7.5MB (量化后, 与 detector 合计)
 - 框架: TensorFlow Lite Micro (TFLM)
- 手势分类器: INT8 MLP 时序分类器
 - 输入: 32 帧 × 21×3 关键点 = 2016 维向量
 - 网络结构: 2016 → 128 → 64 → 50 (类别数)
 - 量化: 全 INT8 量化, 模型大小 270KB
 - 权重格式: 自定义二进制 (magic + 版本 + 层参数 + INT8 权重 /float32 偏置), 板端加载时做全量边界校验

2. 关键机制设计 滑动窗口机制:

- 32 帧滑动窗口, 带回绕解绕处理
- 窗口移除最早帧, 插入最新帧
- 支持连续手势识别

置信度过滤:

- 识别结果置信度阈值: 0.7
- 低于阈值的结果不输出, 避免误识别
- 连续 N 帧相同结果才输出, 提高稳定性

状态机设计:

IDLE → DETECTING → RECOGNIZING → RESULT → IDLE

- IDLE: 空闲状态, 等待检测
- DETECTING: 检测到手部, 开始采集帧
- RECOGNIZING: 帧采集完成, 开始推理
- RESULT: 推理完成, 输出结果

3. openvela 系统能力的深度运用 落地的 openvela 组件:

- 图形: LVGL 界面框架、MIPI DSI 驱动、帧缓冲区管理
- AI: TensorFlow Lite Micro 推理框架
- 多媒体: I2S 音频驱动、nxplayer 播放器、ROMFS 文件系统
- 其他: I2C 总线驱动、触摸输入驱动、摄像头驱动

对 openvela 的优化/拓展:

- 完成 ESP32-P4-Function-EV-Board 的首次板级移植 (arch + 3 板 + 35 外设配置)
- 自研 MIPI CSI 驱动 (DW-GDMA 双缓冲取帧 + 帧回调接口)
- gt9xx 驱动增强: 单消息写、4 次重试、限流日志、I2C_RESET 总线 恢复、双地址探测、ioctl 与事件统计 (适配无中断接线场景)
- CMake 构建支持: 顶层多链接脚本 (aliases+memory+sections)、esp-hal-3rdparty 版本对齐与兼容补丁自动应用、espressif mkimage 出 bin 脚本

3.4 系统实现

1. 软件/固件架构 本项目代码分三处交付:

open-vela/nuttx (PR #364, 接续 #347)

```
├── arch/risc-v/src/esp32p4/      # ESP32-P4 芯片支持
│   ├── espressif/               # Espressif HAL (80+ 外设)
│   └── esp32p4_mipi_csi.c       # 自研 MIPI CSI 驱动
```

```

├── boards/risc-v/esp32p4/
│   ├── esp32p4-function-ev-board/    # 目标板 (35 个验证配置)
│   ├── esp32p4-tab5/                 # 参考板
│   └── esp32p4-pico-wifi-wareshare/   # 参考板
├── drivers/input/gt9xx.c              # GT911 触摸驱动增强
└── tools/espressif/espressif_mkimage.cmake # 出 bin 脚本

open-vela/nuttx-apps (PR #123)
├── examples/lvgldemo/                # SignBridge UI + 摄像头预览演示
│   ├── lvgl_demo.c / boot_screen.c  # 主程序 + 开机画面
│   ├── ui_main.c                     # 主界面 (预览/结果/扬声器卡片)
│   ├── camera_module.c               # CSI 取帧 → LVGL canvas 预览
│   ├── audio_beep.c                  # 提示音接口 (与 UI 解耦)
│   └── zh_font_16.c / zh_font_40.c  # 中文字库

```

本仓库 (contest2026_430)

```

├── app/signbridge/                   # 推理应用 (12 个模块)
│   ├── signbridge_main.c / _sm.c    # 入口 + 状态机
│   ├── signbridge_infer.c / _tflm.cc # TFLM 推理管线
│   ├── signbridge_cls_mlp.c          # INT8 MLP 分类器 (ROMFS 加载)
│   ├── signbridge_camera.c           # 帧源抽象 (测试图案 / CSI)
│   ├── signbridge_ui.c / _anim.c     # 界面 + 手语骨架动画
│   ├── signbridge_voice.c / _audio_in.c # 播报 + 采集/VAD
│   ├── signbridge_correct.c          # 离线语义纠错
│   └── signbridge_pm.c / _vocab.c    # 功耗管理 + 50 词共享词表
├── models/                           # hand landmark 模型 + 分类器权重
├── media/signs/                       # 50 词播报 WAV
└── tools/                             # PC 端训练脚本

```

两条应用线的关系: `app/signbridge/` 是完整推理应用 (模型加载、时序分类、语音播报、纠错), 作为系统入口 `CONFIG_INIT_ENTRYPOINT` 开机自启; `examples/lvgldemo/` 是 UI 与摄像头预览的公共仓演示 (开机画面、CSI 实时取帧渲染到 canvas), 二者共用同一套底层驱动, `lvgl_demo` 的 `camera` 模块验证了 CSI 数据面, `signbridge` 的帧源抽象可直接切换到同一通路。

2. 数据流与关键流程设计 启动流程:

系统启动 → 板级初始化 → LVGL 初始化 → 显示开机画面 (3s)
 → 加载模型 (ROMFS) → 初始化摄像头/音频/触摸
 → 进入主界面 → 主循环 (状态机 + LVGL 定时器)

识别流程:

摄像头采集帧 → 手部检测 → 关键点提取
 → 滑动窗口管理 → MLP 分类 → 置信度过滤
 → 结果输出 (显示 + 语音播报)

3. 硬件设计与适配 开发板: ESP32-P4-Function-EV-Board (Espressif 官方评估板)

外接模块:

- 7 寸 MIPI DSI 显示屏 (EK79007AD 1024×600)
- SC2336 2MP 摄像头 (MIPI CSI)
- ES8311 音频 codec + NS4150 功放
- GT911 电容触摸屏

接口接线:

接口	引脚	说明
I2C0 SCL/SDA	GPIO8 / GPIO7	触摸/音频/摄像头共用
I2S0 BCLK/WS	GPIO12 / GPIO10	音频位时钟/字选择
I2S0 MCLK	GPIO13	音频主时钟
I2S0 DOUT/DIN	GPIO9 / GPIO11	音频数据输出/输入
背光	GPIO26	显示屏背光控制
屏复位	GPIO27	显示屏复位
功放使能	GPIO53	NS4150 使能

关键 BOM:

部件	型号	数量
主控板	ESP32-P4-Function-EV-Board	1
显示屏	7 寸 MIPI DSI 1024×600	1
摄像头	SC2336 2MP	1
音频 codec	ES8311	1
功放	NS4150	1
触摸屏	GT911 电容触摸	1

新硬件适配:

- 是否完成全新硬件平台适配: 是
- 芯片/开发板型号: ESP32-P4 / ESP32-P4-Function-EV-Board
- 驱动类型: MIPI DSI 显示驱动、MIPI CSI 摄像头驱动、I2C 触摸 驱动、I2S 音频驱动、CMake 构建系统
- 适配难点与解决方案:
 1. **ESP32-P4** 在 **openvela** 上无现成支持 → 参考 Espressif 官方 HAL 库实现 NuttX 适配层, 覆盖 80+ 外设
 2. **MIPI CSI** 驱动需自研 (NuttX 无标准接口) → 基于 DW-GDMA 实现双缓冲取帧 + 帧回调
 3. **GT911 INT/RESET** 未接主控 → 轮询模式 (irq 置空, read 直读 状态寄存器), 实测教训 沉淀进驱动: ack 重试会把 GT911 打入 永久 NACK, 只能单次尝试
 4. **I2C0** 单总线挂三设备 → NuttX 总线锁机制, 400kHz 统一时钟
 5. 构建系统: CMake 侧 HAL 版本不一致、漏打兼容补丁、链接 脚本仅支持单脚本 → 版本对 齐 b90b1837、克隆后自动打补丁、多脚本逐个 -T, 最终全链路出 bin
 6. 深睡模式受限 → 触摸中断缺失无可靠唤醒源, 当前仅背光级 功耗管理, 待硬件改线后补 齐

4. 应用/交互端设计 UI 设计:

- 主界面: 左侧摄像头预览区 (实时画面), 右侧上部识别结果卡片 (文字 + 置信度进度条), 右侧下部扬声器按钮
- 开机画面: 白色背景 + openvela logo + 底部进度条 + “正在启动” 状态文字 (3 秒)
- 配色: 浅色卡片风格, 圆角设计
- 中文字库: 16/40px (lv_font_conv 生成, 全量符号集 + FontAwesome 图标 fallback)

交互流程:

1. 开机 → 显示开机画面 → 自动进入主界面
2. 摄像头实时预览 → 检测到手部 → 开始识别
3. 识别完成 → 显示结果文字 + 置信度 → 语音播报
4. 点击扬声器按钮 → 重新播报最近识别结果

3.5 系统测试与结果分析

1. 测试环境

- 硬件：ESP32-P4-Function-EV-Board + 7 寸显示屏 + SC2336 摄像头 + ES8311 音频模块
- 软件：openvela（Apache NuttX）+ LVGL 9.x + TensorFlow Lite Micro
- 工具：串口调试工具、esptool、逻辑分析仪、示波器

2. 构建与静态验证（已全部通过）

验证项	结果
nsh 配置 CMake 全链路构建（HAL 拉取/补丁/编译/链接/mkimage）	通过，nuttx.bin 281KB，0 error
lvgl 配置全量编译（应用 + 模型 + 词库）	通过，产物 10.7MB，0 error
板级外设配置集（35 个：adc/pwm/i2c/spi/ethernet/twai/watchdog 等）	已验证
应用源码 checkpatch	0 error / 0 warning
关键符号入链（入口/状态机/窗口/音频/触摸）	已核对
引脚分配与官方原理图、esp-bsp 一致性	逐项核对（见 board/pins.md）
CSI 采集与 LVGL 预览链路	已实现并编译通过

3. 整机功能测试（待烧录复验）

测试项	预期结果	状态
MIPI DSI 显示	1024×600 正常显示	待烧录复验（编译与配置已核对）
GT911 触摸	触摸响应正常	待烧录复验
摄像头预览	CSI 实时画面	待烧录复验（数据面已实现）
语音播报	扬声器出声	待烧录复验
推理端到端	摄像头 → 模型 → 识别结果	待烧录复验（链路已入链）

4. 性能指标（设计目标，待烧录实测）

指标	目标值	说明
推理帧率	≥10 FPS	需端到端实测
推理时延	≤100ms	需端到端实测
显示刷新率	≥30 FPS	LVGL 定时器配置
触摸响应延迟	≤50ms	轮询模式
功耗（工作）	≤2W	需实测

漏检兜底方案：

- 置信度低于阈值时显示“未识别”提示
- 连续 N 帧无识别结果时提示用户调整手势
- 支持手动触发重新识别

3.6 AI-Native 开发说明

指标	说明
AI Coding 方式 使用的 AI 工具 日志归集	全程 AI 辅助开发（板级移植、驱动、应用、构建链、文档） opencode 等，会话记录见 logs/（qoder-signbridge-20260821，124 事件） 由组委会提供的日志归集工具导出，JSONL 逐事件提交

AI 工具对开发效率的提升：

- 板级移植期：AI 辅助定位构建错误（fork 与上游 API 差异、esp-hal 版本锁定、CMake 链接脚本组织），快速收敛到可出 bin 的构建链
- 触摸接线与 I2C 地址考证：AI 辅助分析原理图和 esp-bsp 源码，确认轮询模式可行
- 模型量化格式设计：AI 辅助设计分类器权重二进制格式，避免板端 越界读
- checkpatch 合规修复：AI 辅助批量修复代码风格问题

遇到的问题与解决方式：

1. 构建错误：fork 与上游 API 差异导致编译失败 → AI 辅助分析 错误日志，定位到 API 变更点
2. 触摸接线确认：GT911 INT/RESET 未接主控 → AI 辅助分析原理图 和 esp-bsp 源码，确认轮询模式可行
3. 模型格式设计：分类器权重加载可能越界 → AI 辅助设计带边界 校验的二进制格式

3.7 总结与展望

1. 成果总结 已完成：

- 硬件适配：ESP32-P4-Function-EV-Board 在 openvela 上的首次 板级移植，含 MIPI DSI 显示、MIPI CSI 摄像头、GT911 触摸、CMake 构建链，全部通过编译与静态验证
- 应用开发：signbridge 推理应用（状态机/推理/界面/播报/纠错）与 lvglDemo UI 摄像头演示双线程交付
- 模型集成：MediaPipe TFLite 手部关键点模型 + INT8 MLP 分类器（自定义权重格式 + 边界校验加载）

当前状态：编译、静态检查、引脚核对全部通过；整机链路待烧录 复验——这是当前唯一的关键路径。

2. 应用前景与商业价值 目标受众：

- 听障人士及其家庭（全球约 4.66 亿听障人士）
- 手语学习者
- 公共服务场所（医院、银行、政务大厅）
- 教育机构（特殊教育学校）

商业模式：

- 硬件销售：便携式手语翻译终端
- 软件授权：手语识别算法授权给手机厂商、教育机构
- 云服务：语义纠错与补全增值服务（订阅制）

规模化潜力：

- 技术可移植到其他 RISC-V 平台，降低硬件成本
- 手语库可持续扩展，覆盖更多词汇
- 可集成到智能眼镜、智能手表等穿戴设备

3. 不足与未来工作 当前不足:

- 整机链路待烧录复验, 性能指标需实测
- 唤醒词模型未训练, 语音唤醒由 VAD 兜底
- 云端纠错仅预留接口, 未实现应用层
- 深睡低功耗未实现 (依赖硬件改线)

未来工作:

1. 短期: 完成烧录复验与性能实测; 训练唤醒词模型
2. 中期: 优化推理帧率; 扩展手语词库; 补深睡低功耗
3. 长期: 移植到其他 RISC-V 平台; 多设备协同; 商业化探索